

Java Developer Assessment – Simple Chat Platform

Overview

The purpose of this assessment is to evaluate a Java developer's ability to design and implement a simple, clean, and well-structured system using a modern Java backend, an Angular frontend, and an asynchronous messaging mechanism.

This assessment is intentionally lightweight and does not require a database or production-level features. We are primarily interested in code quality, design decisions, and understanding of messaging concepts.

Business Case

User Perspective

As a user, I want to:

- Enter a username
- Join a chat platform
- Send messages
- See messages from other users in real time

System Perspective

As a system, I want to:

- Handle multiple users chatting at the same time
 - Process chat messages asynchronously
 - Broadcast messages to all connected users
-

Functional Requirements

Chat Functionality

- User enters a username (no authentication required)
- User can join a default chat room
- User can send chat messages
- User can receive messages sent by other users

Each message should include:

- Username

- Message content
- Timestamp

Messaging Behavior

- Messages must be placed onto a message queue
 - Messages must be processed asynchronously
 - Processed messages must be broadcast to all connected users
-

Data Storage

- No database is allowed
 - All data must be stored in memory (e.g. Lists, Maps, Queues)
 - Chat history may be lost when the application restarts
-

Technical Requirements

Frontend

- Angular
- Simple UI (basic CSS or Bootstrap is acceptable)
- Communication with backend via REST, WebSockets, or both

Backend

- Java (Java 17 or newer preferred)
- Spring Boot
- Clear separation of layers:
 - Controller / API layer
 - Service layer
 - Messaging layer

Build

- Apache Maven ((highly recommended)

Messaging Queue

Candidates may choose **one** of the following:

- Java in-memory queue (e.g. BlockingQueue)
- Spring Application Events
- Embedded or lightweight message broker

Simplicity is preferred. Over-engineering is discouraged.

Architecture & Design Guidelines

- Clean and readable code
- Meaningful naming conventions
- SOLID principles
- Dependency Injection
- Clear separation of responsibilities

The candidate should be able to **clearly explain their design decisions**.

Non-Functional Requirements

- Application must compile and run
 - All components must work together
 - A README file must be provided with:
 - Steps to start the backend
 - Steps to start the frontend
 - Brief explanation of the solution architecture
-

What We Will Be Evaluating

1. Code Quality

- Structure and organization
- Readability and maintainability
- Separation of concerns

2. Messaging Understanding

- Correct usage of asynchronous messaging
- Clear producer and consumer logic
- Understanding of message flow

3. Java Fundamentals

- Proper use of Spring and dependency injection
- Clear service and controller responsibilities

4. Frontend Integration

- Ability of Angular application to send and receive messages
- Correct consumption of backend APIs or WebSocket endpoints

5. Communication Skills

- Ability to walk through the code
 - Explanation of design choices and trade-offs
-

Optional Enhancements (Not Required)

These are **nice-to-have only** and will not negatively impact scoring if omitted:

- Basic input validation
 - Multiple chat rooms
 - Simple unit tests
 - Improved UI polish
-

Out of Scope

- Authentication and authorization
 - Persistent storage
 - Advanced security features
 - Production scalability
-

Assessment Instructions

- Candidates should implement the solution independently
 - Build time recommendation: 1–2 days
 - Be prepared to present and explain the solution during a walkthrough
-

End of Assessment Document